Alternative Project Part 5 - Database Project

Cecilia Smith

ITD 256

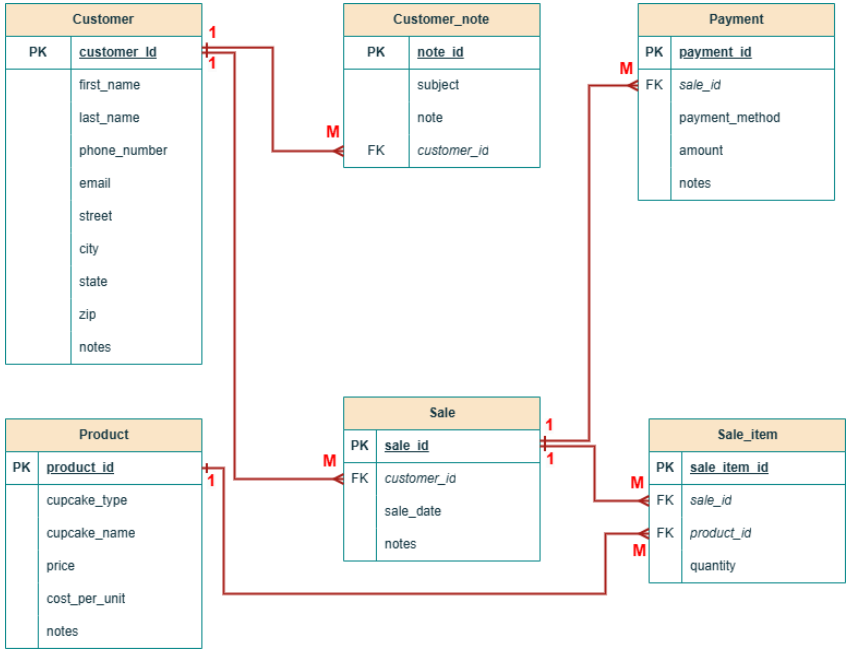
May 4, 2025

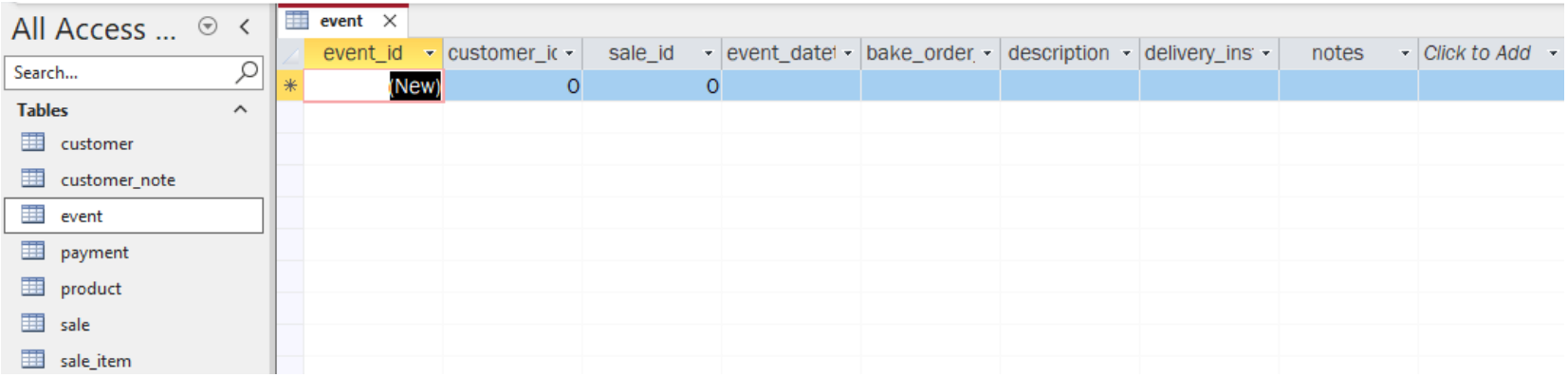
Introduction

- This project focuses on designing and managing a relational database for a cupcake shop using SQL and best practices in database design.

ERD Overview

- The Entity-Relationship Diagram (ERD) represents the database structure for the cupcake shop, highlighting relationships between key tables: customer, customer_note, product, sale, payment, and sale_item. The design reflects the project schema as implemented.

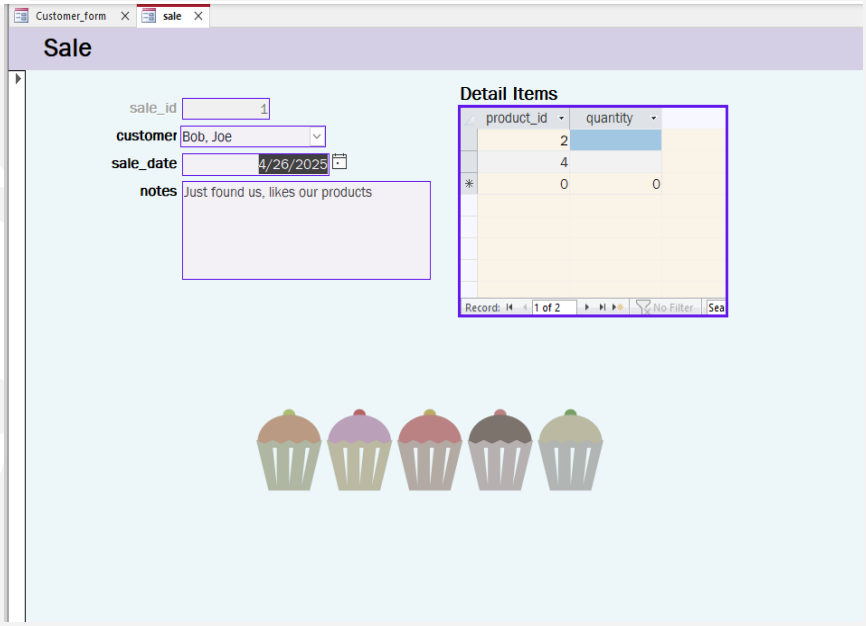


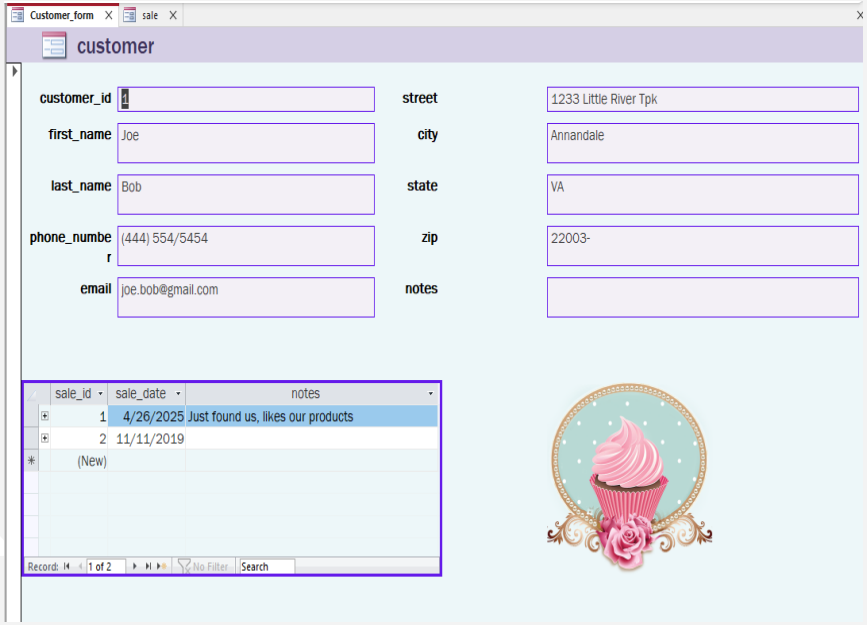


Access Database: Schema and Interface Enhancements

- The Access database updates include modifications to the schema, such as adding a quantity field to the sale items and a unit capacity per day field to the product table. A new table for event catering was also created, featuring fields for customer and sale IDs, event details, and delivery instructions.

- On the user interface side, a new customer form was designed with a combo box for easier selection, along with a sale form that includes a subform displaying related sale items. These updates aim to improve functionality and data management within the system.





MySQL Database Schema

```
create database annandale_cupcakes;
   use annandale_cupcakes;


create table customer (
       customer_id int primary key auto_increment,
       first_name varchar(50),
       last_name varchar(50),
       phone_number varchar(20),
       email varchar(30),
       street varchar(100),
       city varchar(50),
       state varchar(25),
       zip varchar(10),
       notes varchar(255),
       created_at datetime,
       updated_at datetime
   );


CREATE TABLE customer_note (
       note_id INT PRIMARY KEY AUTO_INCREMENT,
       subject VARCHAR(100),
       note TEXT,
       customer_id INT,
       created_at DATETIME,
       updated_at DATETIME,
       FOREIGN KEY (customer_id) REFERENCES
   customer(customer_id)
   );


• CREATE TABLE product (
       product_id INT PRIMARY KEY AUTO_INCREMENT,
       cupcake_type VARCHAR(50),
       cupcake_name VARCHAR(100),
       price DECIMAL(10, 2),
       cost_per_unit DECIMAL(10, 2),
       notes TEXT,
       unit_capacity_per_day INT,
       created_at DATETIME,
       updated_at DATETIME
   );
```

```
CREATE TABLE sale (
       sale_id INT PRIMARY KEY AUTO_INCREMENT,
       customer_id INT,
       sale_date DATE,
       notes TEXT,
       created_at DATETIME,
       updated_at DATETIME,
       FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
   );


CREATE TABLE event (
       event_id INT PRIMARY KEY AUTO_INCREMENT,
       customer_id INT,
       sale_id INT,
       event_datetime DATETIME,
       bake_order_datetime DATETIME,
       description TEXT,
       delivery_instructions TEXT,
       notes TEXT,
       FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
       FOREIGN KEY (sale_id) REFERENCES sale(sale_id)
   );


CREATE TABLE payment (
       payment_id INT PRIMARY KEY AUTO_INCREMENT,
       payment_method VARCHAR(50),
       amount DECIMAL(10, 2),
       notes TEXT,
       sale_id INT,
       created_at DATETIME,
       updated_at DATETIME,
       FOREIGN KEY (sale_id) REFERENCES sale(sale_id)
   );


• CREATE TABLE sale_item (
       sale_item_id INT PRIMARY KEY AUTO_INCREMENT,
       sale_id INT,
       product_id INT,
       quantity INT,
       created_at DATETIME,
       updated_at DATETIME,
       FOREIGN KEY (sale_id) REFERENCES sale(sale_id),
       FOREIGN KEY (product_id) REFERENCES product(product_id)
   );
```

- The MySQL database schema establishes a comprehensive structure for the cupcake business. It includes tables for customers, products, sales, events, payments, and sale items, with clearly defined relationships and constraints to ensure data integrity and support business operations efficiently.

Database Implementation and Queries

```sql
# 1
SELECT customers.name, orders.order_id, orders.product
FROM customers
JOIN orders USING (customer_id);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	order_id	product
Alice	101	Laptop
Bob	102	Smartphone
Bob	103	Keyboard

```sql
# 2
SELECT customers.name, orders.product
FROM customers
LEFT JOIN orders USING (customer_id);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	product
Alice	Laptop
Bob	Keyboard
Bob	Smartphone
Charlie	NULL
Diana	NULL

```sql
# 3
SELECT products.product_name, customers.name
FROM products
LEFT JOIN orders ON products.product_name = orders.product
LEFT JOIN customers ON orders.customer_id = customers.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	name
Laptop	Alice
Smartphone	Bob
Keyboard	Bob
Monitor	NULL

```sql
# 5
SELECT customers.name, orders.product,
    CASE
        WHEN orders.product IN ('Laptop', 'Smartphone') THEN 'High-End'
        ELSE 'Standard'
    END AS product_category
FROM customers
JOIN orders ON customers.customer_id = orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	product	product_category
Alice	Laptop	High-End
Bob	Smartphone	High-End
Bob	Keyboard	Standard

```sql
# 4
SELECT customers.name, orders.product
FROM customers
LEFT JOIN orders ON customers.customer_id = orders.customer_id
UNION
SELECT customers.name, orders.product
FROM orders
LEFT JOIN customers ON customers.customer_id = orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	product
Alice	Laptop
Bob	Keyboard
Bob	Smartphone
Charlie	NULL
Diana	NULL
NULL	Monitor

```
57 • DESCRIBE orders;
58
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
customer_id	int	YES		NULL	
product	varchar(50)	YES		NULL	

```
54 -- Use the describe command
55 • DESCRIBE customers;
56
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	

```
54 • DESCRIBE products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	
product_name	varchar(50)	YES		NULL	

```
55 • DESCRIBE payments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	
order_id	int	YES		NULL	
amount	decimal(10,2)	YES		NULL	

Description:

For this part of the project, I worked on creating and populating four tables in MySQL Workbench: customers, orders, products, and payments. These tables were designed to simulate real-world business scenarios with structured data.

The task included the following:

1. Creating the tables using the provided schema and inserting the sample data.
2. Using the **DESCRIBE** command to display the structure of each table.
3. Executing specific queries to analyze relationships between tables and generate reports.

The queries addressed questions like:

- Listing customers with their orders.
- Displaying all customers, whether or not they have placed orders.
- Identifying ordered products and matching them to customers.
- Reporting unmatched records across tables.
- Categorizing products as "High-End" or "Standard" based on type.

Conclusion

- The project successfully implemented a database system with well-defined relationships and effective SQL queries, ensuring functional data organization and integrity suitable for the application requirements.

References

- SAKILA SCHEMA DOCUMENTATION

- COURSE MATERIALS AND TUTORIALS

- ONLINE SQL RESOURCES